# Module 3: End of Module Assignment

## TaskFlow - Task Management API

### Overview

**Type:** Mandatory Assignment

**Estimated Time:** 2-3 hours

### Introduction

Build a simple TaskFlow API, a task management backend. You will create a Spring Boot project, build it using the layered structure you have learned, and push it to GitHub.

There is no code given in this assignment. Write it yourself using what you have learned in class.

---

## Project Setup

### Step 1: Create Spring Boot Project

1. Go to https://start.spring.io/
2. Configure:
    - Project: Maven
    - Language: Java
    - Spring Boot: current stable release offered by Spring Initializr
    - Group: sg.edu.ntu
    - Artifact: taskflow-api
    - Packaging: Jar
    - Java: 21
3. Add Dependencies:
    - Spring Web
    - Lombok
    - Spring Boot DevTools
    - OpenAI (this is the Spring AI starter, needed only if you attempt Task 6)
4. Click "Generate" and download
5. Extract the ZIP file
6. Open the extracted folder in VS Code

---

## Your Tasks

### TASK 1: Create the Folder Structure

Inside src/main/java/sg/edu/ntu/taskflowapi/, create four folders:

- controller

- model
- service
- repository

Every class you write goes into the folder that matches its layer.

---

### TASK 2: Model Layer

Create a Task class in the model folder.

- It is a plain POJO. Do not add any Spring annotations to it.
- Decide your own fields. At minimum it should have an id, a title, and a completed flag.
- Use Lombok for getters, setters and constructors.

---

### TASK 3: Repository Layer

Create a TaskRepository class in the repository folder and annotate it with @Repository.

- Store the tasks in memory. Choose either an ArrayList or a HashMap.
- If you use an ArrayList, you will need to loop through the list to find a task by its id. Write a small helper method for this.
- If you use a HashMap, you can look a task up directly by its key.
- Generate your own ids. A simple counter is enough.
- Initialise your collection so it is never null.

---

### TASK 4: Service Layer

Create a TaskService class in the service folder and annotate it with @Service.

The service holds your business logic and calls the repository. Write these methods:

- findAllTasks()
- findTaskById(Long id)
- createTask(Task task)
- updateTask(Long id, Task task)
- deleteTask(Long id)
- markTaskAsComplete(Long id)

For markTaskAsComplete, find the task, set its completed flag to true, and save it back.

Use constructor injection to bring the repository into the service.

---

### TASK 5: Controller Layer

Create a TaskController class in the controller folder. Annotate it with @RestController and map it to /api/tasks.

Build these endpoints:

| Method | Path | What it does |
|---|---|---|
| GET | /api/tasks | Get all tasks |
| GET | /api/tasks/{id} | Get one task by id |
| POST | /api/tasks | Create a new task |
| PUT | /api/tasks/{id} | Update an existing task |
| DELETE | /api/tasks/{id} | Delete a task |
| PUT | /api/tasks/{id}/complete | Mark a task as complete |

Return the correct status codes. 201 for create, 200 for the rest.

Use constructor injection to bring the service into the controller.

---

### TASK 6 (Additional Step): Spring AI Summary Endpoint

**This step is optional. Tasks 1 to 5 are enough to pass. Complete this one for a better score.**

Add one more endpoint that uses Spring AI.

- Create a ChatClient and inject it where you need it.
- Add an endpoint at GET /api/tasks/summary.
- It should take your list of tasks, send it to the model, and return a short plain-English summary of what is pending and what is done.

**Your API key must not be hard coded.**

- Keep the key in an environment variable in ~/.bashrc, the same way we did in class.
- In application.properties, refer to it like this: spring.ai.openai.api-key=${OPENAI_API_KEY}
- That line is safe to push to GitHub because it holds no key, only the variable name. Spring reads the real value from your environment when the app starts.
- The actual key must not appear anywhere in your project files.

---

## Testing Your Work

1. Run the application:

```
mvn spring-boot:run
```

2. Use Postman to test every endpoint you built.
3. Create a few tasks first, then test get, update, delete and complete. Test summary too if you attempted Task 6.

## Submission

### Step 1: Create GitHub Repository

1. Go to github.com
2. Click "New repository"
3. Name it: module3-taskflow-api (or any name)
4. Make it PUBLIC
5. Click "Create repository"

### Step 2: Push Your Code

Open the Terminal in VS Code (View → Terminal) and make sure you are inside your taskflow-api project folder.

Run these commands one by one:

```
git init
git add .
git commit -m "Complete Module 3 assignment"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/YOUR-REPO-NAME.git
git push -u origin main
```

Replace YOUR-USERNAME and YOUR-REPO-NAME with your actual values.

**Important:** if you attempted Task 6, check that your application.properties uses ${OPENAI_API_KEY} and not the actual key before you push.

### Step 3: Submit to NTU Learn

1. Copy your GitHub repository URL
2. Submit it in the assignment submission on NTU Learn

## Done!